

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION
TECHNOLOGY



SOICT

PARALLEL AND DISTRIBUTED PROGRAMMING REPORT:

MPI Sinkhorn Optimal Transport Solver

Row-wise Data Decomposition on a Physical VM Cluster

Course:

IT4130E – Parallel and Distributed Programming

Presented by:

Nguyễn Tất Thanh - 20235560

Đặng Thịnh Tường Minh - 20235528

Nguyễn Ngọc Minh - 20235533

Kiều Sơn Tùng - 20235571

Hanoi - Vietnam

2026

This report presents the design, implementation, cluster setup, and benchmark analysis of a C++/MPI Sinkhorn-Knopp solver for entropy-regularized optimal transport on a physical-machine Ubuntu VM cluster.

Contents

1	Project Objective and Requirement Coverage	4
1.1	Team Work Allocation	4
2	Problem and Sequential Algorithm	5
3	Parallel Algorithm Design	5
3.1	Parallelism Type	5
3.2	Communication and Topology	5
3.3	Pseudocode	6
3.4	Why Row Decomposition Was Chosen	6
4	Implementation	7
5	Cluster Setup and Router Evidence	7
5.1	Node Hardware and Router Specifications	7
5.2	Physical Cluster Layout	8
5.3	Router Setup Procedure	9
5.4	MPI Hostfile and Smoke Test	9
6	Correctness Validation	10
7	Experimental Plan	10
8	Baseline Router and Wi-Fi Results	10
8.1	Input-Size Sweep	10
8.2	Process Sweep and Efficiency	11
9	Required Calibration and Load-Balance Results	12
9.1	Two- to Three-Minute Workload	12
9.2	Per-Rank Load Balance	13
10	Speedup at the $2N = 16000$ Input	14
11	Short Python Prototype Comparison	14
12	Network and Wireless-Band Experiments	15
12.1	Two-Node Same-Band Wi-Fi versus Phone Hotspot	15
12.2	Communication-Reduction Variants	16

13 Four-Machine Wi-Fi Experiment	17
14 Dynamic Scheduling Experiments	18
14.1 Implemented Dynamic Modes	18
14.2 Does Dynamic Scheduling Improve Performance?	18
15 Discussion	19
15.1 What the Experiments Show	19
15.2 Limitations	20
15.3 Future Work	20
16 Conclusion	20

1 Project Objective and Requirement Coverage

The course task requires a group of four students to build a real MPI program in C/C++, run it on at least three physical machines, and analyze its parallel behavior. The selected problem is entropy-regularized optimal transport solved by the Sinkhorn-Knopp algorithm. This is a non-trivial dense numerical workload because each iteration performs matrix-vector style computation over an $N \times N$ kernel matrix and must synchronize a global column vector.

Requirement from task file	How this project satisfies it
At least three physical machines	Benchmarks were run on three-machine router/Ethernet and three-machine Wi-Fi clusters; earlier Wi-Fi runs also include a four-machine cluster.
C/C++ implementation	The main solver is implemented in <code>mpi-sinkhorn-step/cpp/sinkhorn_cpp.cpp</code> .
Demoable MPI program	The executable supports <code>-mode sequential</code> and <code>-mode mpi</code> ; MPI runs use OpenMPI hostfiles and SSH.
Parallelism type and decomposition	Data-level parallelism with one-dimensional row-wise data decomposition.
Mapping and communication strategy	Rows are mapped to ranks; each iteration uses local dense compute and <code>MPI_Allreduce</code> for the global column update.
Load-balance analysis	Per-rank compute and communication times are exported to JSON and summarized in the report.
Correctness validation	MPI objective/error metrics are compared against sequential output with the same random seed and iteration count.
Runtime, speedup, efficiency	The report includes size sweep, process sweep, workload calibration, $2N$ speedup, router-vs-Wi-Fi, and dynamic scheduling experiments.

Table 1: Mapping from assignment requirements to project deliverables.

1.1 Team Work Allocation

Member	Student ID	Assigned work
Nguyễn Tất Thanh	20235560	Cluster setup, SSH/OpenMPI configuration, router evidence collection
Đặng Thịnh Tường Minh	20235528	Sequential baseline, correctness validation, Python vs C++ comparison
Nguyễn Ngọc Minh	20235533	Load-balance analysis, compute/communication metrics, result visualization
Kiều Sơn Tùng	20235571	MPI implementation, row-wise decomposition, benchmark run scripts

Table 2: Team work allocation for implementation, experiments, and result analysis.

2 Problem and Sequential Algorithm

Optimal transport searches for a transport matrix P that moves a source distribution a to a target distribution b while minimizing a cost matrix C . The entropy-regularized version adds a smoothing term and can be solved efficiently by Sinkhorn-Knopp iterations. Given $\varepsilon > 0$, define the kernel

$$K_{ij} = \exp(-C_{ij}/\varepsilon).$$

The solver alternates between two scaling vectors:

$$u = a \oslash (Kv), \quad v = b \oslash (K^T u).$$

The final transport plan is implicitly represented as

$$P = \text{diag}(u)K\text{diag}(v).$$

The sequential algorithm stores the full dense kernel matrix and performs the following work in each iteration:

1. Compute every denominator $(Kv)_i$ and update u_i .
2. Compute every denominator $(K^T u)_j$ and update v_j .
3. Optionally compute row error, column error, and transport objective for validation.

For an $N \times N$ dense matrix, the main computational cost is $O(N^2)$ per iteration and memory usage is also $O(N^2)$. This explains why larger sizes provide more parallel work but also stress VM memory.

3 Parallel Algorithm Design

3.1 Parallelism Type

The implementation uses **data-level parallelism**. The matrix is decomposed by rows. Rank r owns a contiguous interval $[s_r, e_r)$:

$$\text{local_rows}_r = e_r - s_r.$$

Every rank stores only its local kernel block K_r . The vector v is replicated on all ranks. Rank r can update its local entries of u independently:

$$u_i = a_i / \sum_j K_{ij} v_j, \quad i \in [s_r, e_r).$$

3.2 Communication and Topology

After updating local u , each rank computes a partial column contribution:

$$\text{local_col}_j = \sum_{i=s_r}^{e_r-1} K_{ij} u_i.$$

The global column denominator is obtained with

$$\text{global_col} = \text{MPI_Allreduce_sum}(\text{local_col}).$$

Then every rank can update the replicated vector v :

$$v_j = b_j / \text{global_col}_j.$$

The logical communication pattern is collective rather than master-worker. In practice, OpenMPI chooses an internal reduction algorithm depending on process count and network. The important property for this project is that one length- N vector is reduced in every Sinkhorn iteration. With double precision, the main payload is approximately $8N$ bytes per rank per column synchronization.

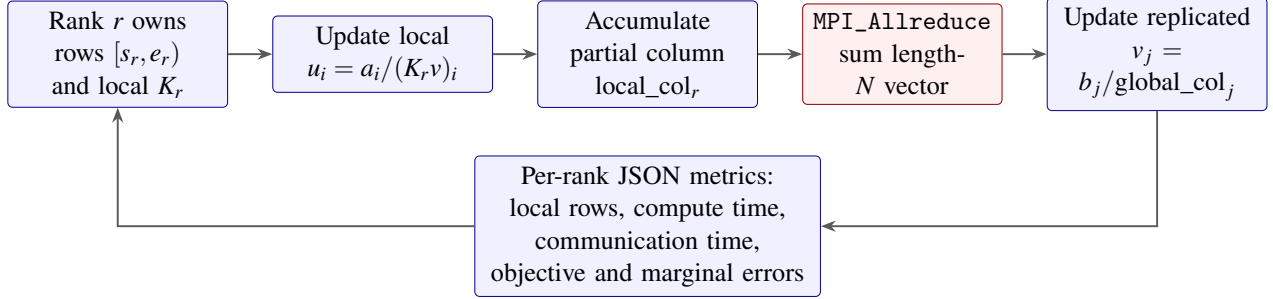


Figure 1: C++/MPI Sinkhorn iteration structure. Computation is row-local; the only mandatory synchronization in each iteration is the column-vector MPI_Allreduce.

3.3 Pseudocode

```

Input: N, epsilon, max_iters, rank r, number of ranks p
Build local row interval [s_r, e_r)
Build local dense kernel K_r
Initialize local u and replicated v

for iter = 1..max_iters:
    for each local row i:
        denom = sum_j K_r[i,j] * v[j]
        u[i] = a[i] / denom

    local_col[:] = 0
    for each local row i:
        for each column j:
            local_col[j] += K_r[i,j] * u[i]

    global_col = MPI_Allreduce_sum(local_col)
    for each column j:
        v[j] = b[j] / global_col[j]

Compute objective/error metrics for validation
Write JSON output with runtime and per-rank metrics
  
```

3.4 Why Row Decomposition Was Chosen

One-dimensional row decomposition was chosen because it is reliable on a small physical cluster and maps directly to the structure of the Sinkhorn update. A two-dimensional block decomposition could reduce per-rank memory and communication at larger scale, but it would require more complex

redistribution of u , v , and partial sums. For this course project, the row-wise design gives a clear explanation, a robust demo, and measurable speedup.

4 Implementation

The C++ implementation is located in `mpi-sinkhorn-step/cpp/sinkhorn_cpp.cpp`. The executable accepts command-line options for matrix size, mode, epsilon, seed, maximum iterations, communication mode, partition mode, and output path.

The C++ path is the main deliverable of the project. It uses explicit flat arrays for the local dense kernel block and vectors, so the inner loops are simple contiguous-memory scans. This matters for the benchmark: most useful work is spent in repeated floating-point loops over K_r , while the MPI layer only exchanges the column denominator vector. The implementation also emits per-rank timing fields, which makes it possible to separate compute imbalance from communication overhead in later experiments.

Component	Role
Sequential solver	Baseline for correctness and speedup.
MPI solver	Row partitioning, local dense compute, MPI_Allreduce, per-rank timing.
Benchmark scripts	Run repeated experiments for sizes, process counts, network setups, and partition modes.
Summarizers	Convert JSON outputs to CSV/Markdown tables and figures.
Dynamic partition modes	Experimental chunk-based scheduling modes for heterogeneous machines.

Table 3: Main implementation components.

Each JSON result contains runtime, objective, row error, column error, hostnames, local rows per rank, rank compute time, rank communication time, and rank total time. These fields are used to compute speedup, efficiency, imbalance, and communication overhead.

5 Cluster Setup and Router Evidence

5.1 Node Hardware and Router Specifications

The cluster is heterogeneous: the virtual machines expose the same number of virtual CPU cores for the main MPI runs, but the physical processors and memory limits differ. Public CPU scores are included only as a rough reference; actual MPI performance also depends on thermal limits, VM scheduling, memory pressure, and network behavior.

Node	Physical CPU	CPU cores / threads	VM CPU	VM RAM during project	Geekbench 6 score	Role in experiments
Master	Intel Core i5-9300H	4 / 8	4 vCPU	4 GB → 8 GB	1291 single / 3737 multi	Master for router and latest Wi-Fi runs
Worker 1	Intel Core i5-9300H	4 / 8	4 vCPU	4 GB → 8 GB	1291 single / 3737 multi	Router/Wi-Fi worker
Worker 2	AMD Ryzen 5 5600H	6 / 12	4 vCPU	3 GB → 6 GB	1594 single / 5716 multi	Router worker, stronger physical CPU but initially memory-limited
Worker 3	Intel Core i3-1115G4	2 / 4	4 vCPU	4 GB → 6 GB	1571 single / 3056 multi	Wi-Fi worker used in the latest three-node Wi-Fi run

Table 4: Physical machines and VM resource allocation. CPU core/thread counts are from Intel/AMD specifications; Geekbench 6 scores are public user-submitted averages.

The router used for the wired/router experiment was a TP-Link Archer C20 AC750 dual-band router. Its relevant specifications are: Wi-Fi 5/802.11ac support, 433 Mbps on 5 GHz, 300 Mbps on 2.4 GHz, three fixed antennas, one 10/100 Mbps WAN port, and four 10/100 Mbps LAN ports. The 100 Mbps Ethernet ports are important: the router setup is more stable than Wi-Fi, but it is not a gigabit wired cluster. This helps explain why communication-heavy MPI runs can still be limited by network synchronization.

Hardware/specification sources: Intel ARK for i5-9300H and i3-1115G4, AMD product specifications for Ryzen 5 5600H, Geekbench Browser for CPU scores, and TP-Link Archer C20 specifications.

5.2 Physical Cluster Layout

The primary router/Ethernet experiment used three physical machines, each running one Ubuntu VM:

- Master: 192.168.1.25
- Worker 1: 192.168.1.26
- Worker 2: 192.168.1.75

The three machines were connected through the same Archer router. The VM adapters were configured in bridged mode so that each Ubuntu VM received an address on the same 192.168.1.0/24 LAN.



Figure 2: Three physical machines connected through one router for the MPI cluster.

5.3 Router Setup Procedure

The router setup used static IPv4 addresses to avoid SSH and MPI host discovery changing between runs. On each Ubuntu VM, the network interface was checked with:

```
ip -br addr
ip route
```

When NetworkManager was unavailable, static addressing was configured through Netplan. A typical master configuration was:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: no
      addresses: [192.168.1.25/24]
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

The workers used the same subnet with different addresses, for example 192.168.1.26/24 and 192.168.1.75/24. After applying Netplan, connectivity was validated with ping and ssh.

5.4 MPI Hostfile and Smoke Test

For the stable router baseline, the hostfile was:

```
192.168.1.25 slots=2
192.168.1.26 slots=2
192.168.1.75 slots=2
```

The basic MPI smoke test was:

```
mpirun -np 3 --hostfile hosts_router_3_slots2 \
  --map-by node \
  --mca btl tcp,self \
  --mca btl_tcp_if_include enp0s3 \
  --mca oob_tcp_if_include enp0s3 \
  hostname
```

The same pattern was used for benchmarks, replacing hostname with sinkhorn_cpp -mode mpi
....

6 Correctness Validation

Correctness was validated by running sequential and MPI modes with the same N , seed, ϵ , cost mode, and iteration count. The MPI result is accepted if objective and marginal errors match the sequential run within floating-point tolerance.

Across the Wi-Fi three-node sweep, the maximum objective difference remains at floating-point scale. For example, at $N = 12000$, the maximum objective difference is 6.448×10^{-14} . This is expected because the MPI reduction changes the order of floating-point addition, but not the mathematical computation.

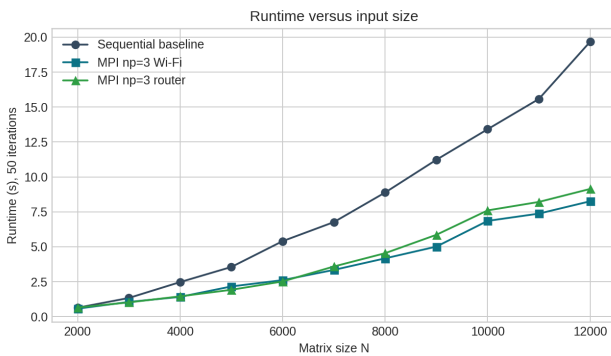
7 Experimental Plan

The experiments follow the requirements in the task file:

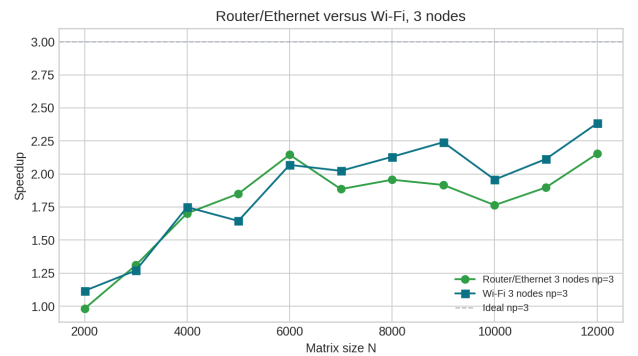
1. **Input-size calibration:** run $N = 2000..12000$ with fixed 50 iterations. Then keep $N = 8000$ and increase iterations to reach a two- to three-minute workload.
2. **Granularity/load-balance analysis:** use the calibrated long workload and plot per-rank compute/communication time.
3. **Speedup analysis:** run the $2N = 16000$ case with process counts 1, 2, 4, and 8.
4. **Network comparison:** compare router/Ethernet, Wi-Fi, same-band Wi-Fi, and phone-hotspot experiments.
5. **Dynamic scheduling:** compare contiguous static partitioning with weighted chunk modes and runtime queue mode.

8 Baseline Router and Wi-Fi Results

8.1 Input-Size Sweep



(a) Runtime versus input size.



(b) Three-node speedup.

Figure 3: Input-size sweep and router/Ethernet versus Wi-Fi comparison with 50 Sinkhorn iterations.

N	Seq avg (s)	Router np3 (s)	Router speedup	Wi-Fi np3 (s)	Wi-Fi speedup
4000	2.471	1.616	1.529	1.413	1.749
8000	8.881	3.945	2.251	4.170	2.130
12000	19.673	8.901	2.210	8.255	2.383

Table 5: Representative router/Ethernet and Wi-Fi results.

The main conclusion is that MPI becomes beneficial when N is large enough. Small sizes are dominated by startup, memory allocation, and communication overhead. At $N = 8000$ – 12000 , both router and Wi-Fi runs reach around $2.1\times$ – $2.4\times$ speedup on three processes.

Router/Ethernet is more stable operationally: it avoided the intermittent Wi-Fi master disappearance observed during one Wi-Fi run. However, raw runtime does not always make router faster, because the machines are heterogeneous and VM scheduling noise is significant. Therefore, the correct conclusion is not “router always wins”, but rather: router is a more reliable demo network, while performance still depends on node speed and run noise.

The best three-node router result observed in an earlier fixed-50 benchmark is stronger than the later safe-suite average: at $N = 12000$, the run `bench_cpp_router3_25_26_75_fixed50_reps4` reports 6.817 s MPI runtime, $2.8859\times$ speedup, and 0.962 efficiency. We treat this as the best observed point, while the later router/Wi-Fi report-suite results are used as the more conservative comparison set.

8.2 Process Sweep and Efficiency

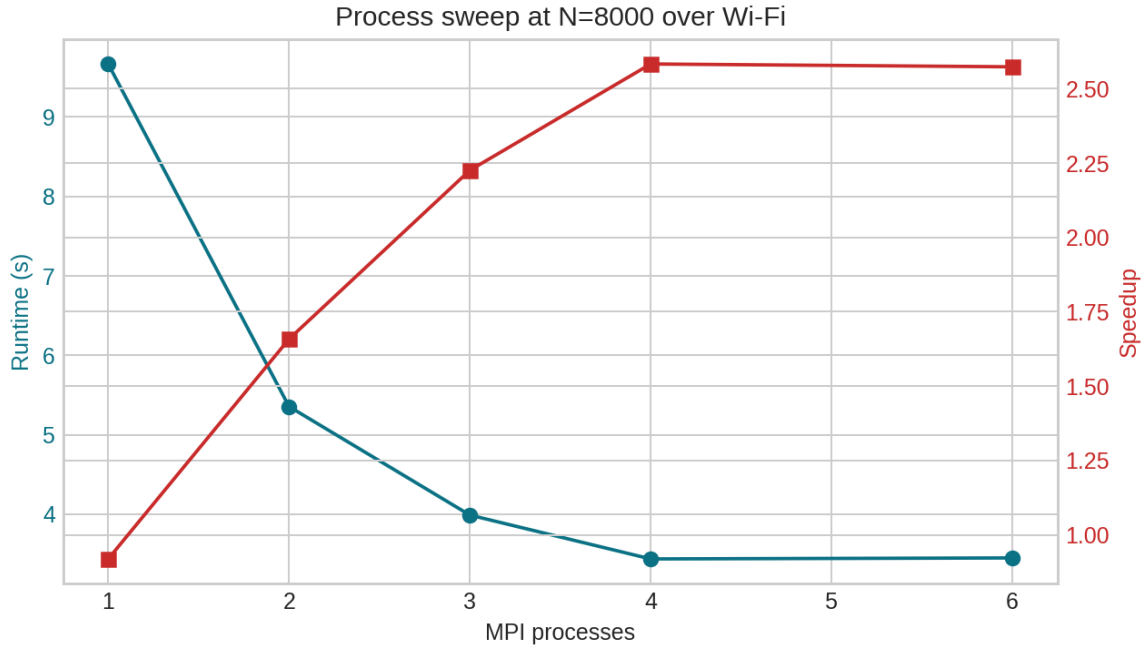


Figure 4: Runtime and speedup when varying process count at $N = 8000$ over Wi-Fi.

Processes	Runtime (s)	Speedup	Efficiency	Avg imbalance (%)
1	9.667	0.919	0.919	0.000
2	5.355	1.658	0.829	0.048
3	3.990	2.226	0.742	0.191
4	3.438	2.583	0.646	0.512
6	3.450	2.574	0.429	0.452

Table 6: Wi-Fi process sweep at $N = 8000$, 50 iterations.

Runtime improves from one to four processes, but six processes do not improve over four. This is the clearest sign of communication overhead: when each rank owns fewer rows, local compute shrinks but the Allreduce vector is still length N in every iteration.

9 Required Calibration and Load-Balance Results

9.1 Two- to Three-Minute Workload

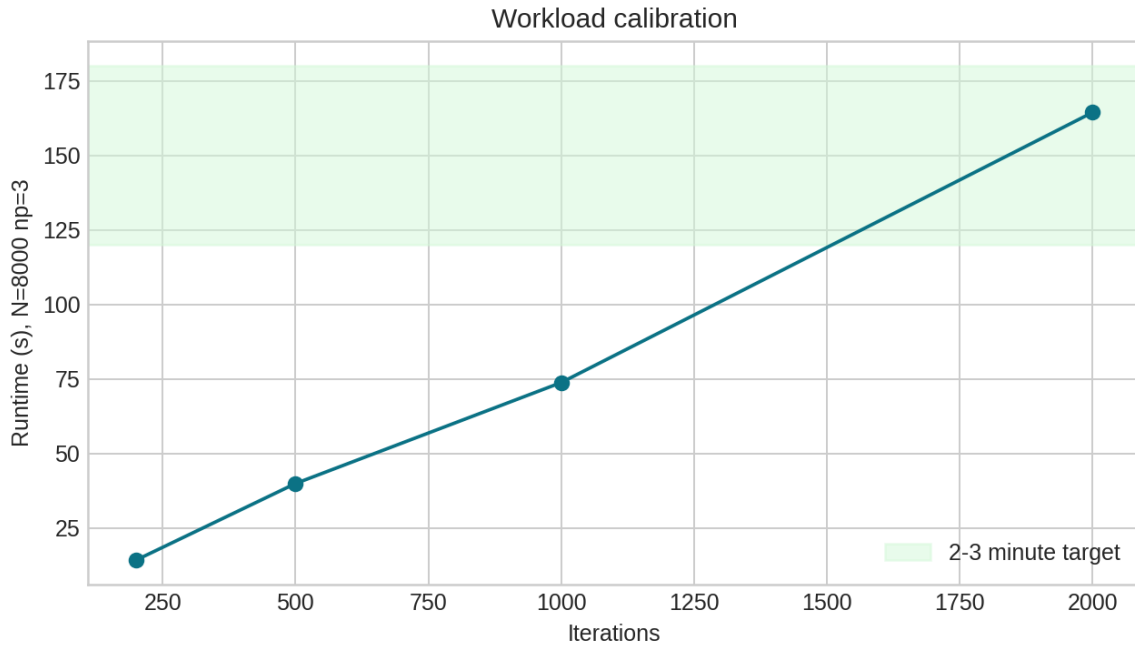


Figure 5: Workload calibration at $N = 8000$, $np=3$.

Iterations	Runtime (s)	Interpretation
200	14.176	Too short
500	39.864	Below one minute
1000	73.746	Around 1.2 minutes
2000	164.394	Within the target two- to three-minute range

Table 7: Long workload calibration.

The task asks for an input size/workload whose full runtime is about two to three minutes. Increasing N too far would push the dense matrix into high RAM pressure, so the safer calibration is $N = 8000$ with 2000 iterations. This reaches 164.394 seconds and is used for the load-balance analysis.

9.2 Per-Rank Load Balance

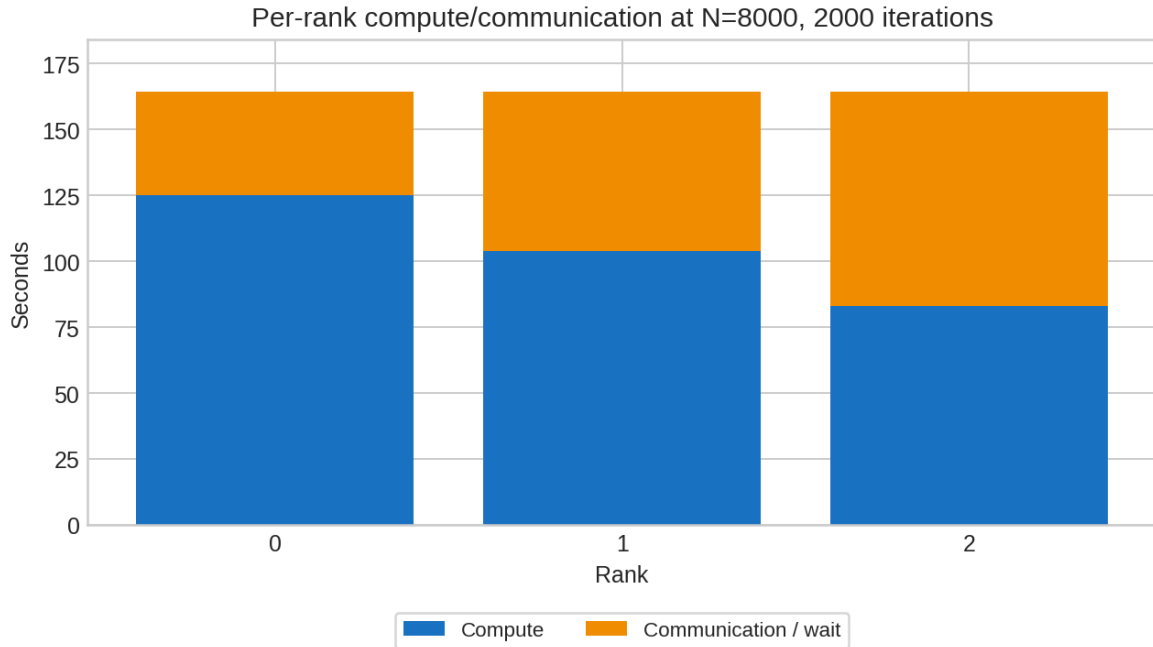


Figure 6: Per-rank compute and communication time on the long workload.

Rank	Local rows	Total (s)	Compute (s)	Communication/wait (s)
0	2667	164.390	125.182	39.189
1	2667	164.394	103.973	60.406
2	2666	164.393	83.085	81.283

Table 8: Per-rank breakdown at $N = 8000$, 2000 iterations, $np=3$.

Total time is almost identical because all ranks synchronize at the collective operation. The compute/communication split reveals heterogeneity: rank 0 computes more and waits less, while rank 2 computes less and waits more. Since total runtime difference is far below 25%, the static contiguous partition satisfies the assignment’s load-balance requirement. Still, this is exactly why dynamic weighted scheduling is worth testing: faster ranks can be assigned more rows.

10 Speedup at the $2N = 16000$ Input

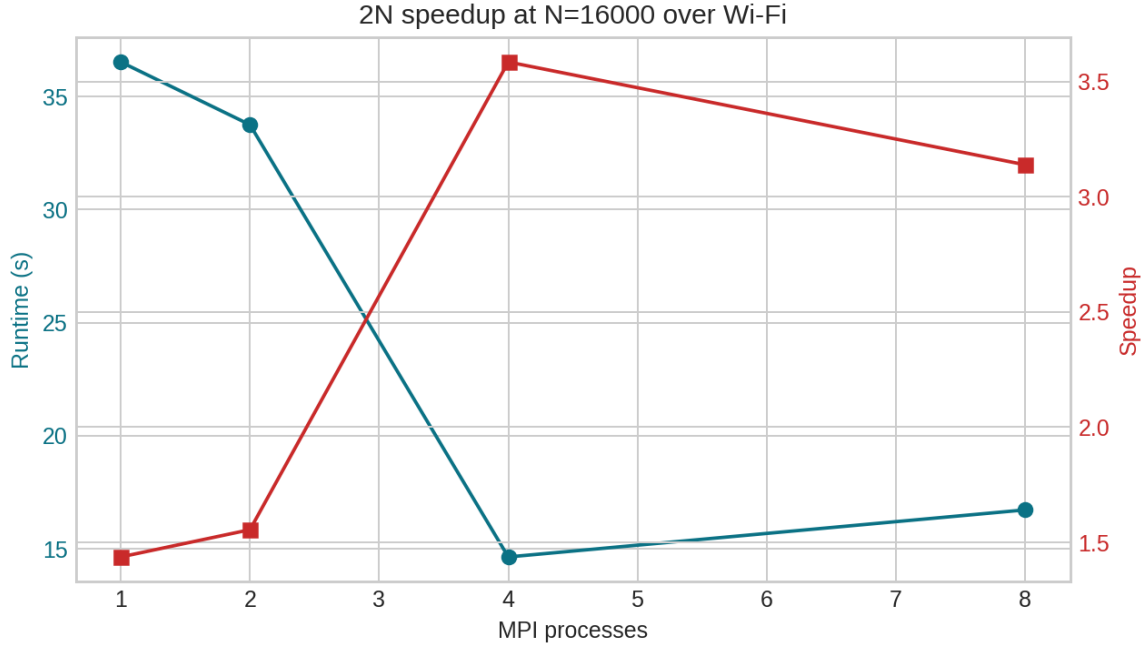


Figure 7: Speedup analysis at $N = 16000$ over Wi-Fi.

Processes	Runtime (s)	Speedup	Efficiency
1	36.525	1.437	1.437
2	33.750	1.555	0.778
4	14.642	3.585	0.896
8	16.721	3.139	0.392

Table 9: Speedup at $N = 16000$, 50 iterations.

The best result is $np=4$, which reaches $3.585\times$ speedup. Increasing to $np=8$ makes the run slower. This confirms the communication bottleneck: beyond a certain process count, the reduction of a length- N column vector costs more than the saved local compute.

11 Short Python Prototype Comparison

The project started with a Python/mpi4py MVP because it was faster to validate the mathematical algorithm and correctness checks. This prototype is not the main implementation. The final benchmark path uses C++/MPI because the assignment requires C/C++, and because C++ gives direct control over memory layout, per-rank timing, communication modes, and dynamic partitioning.

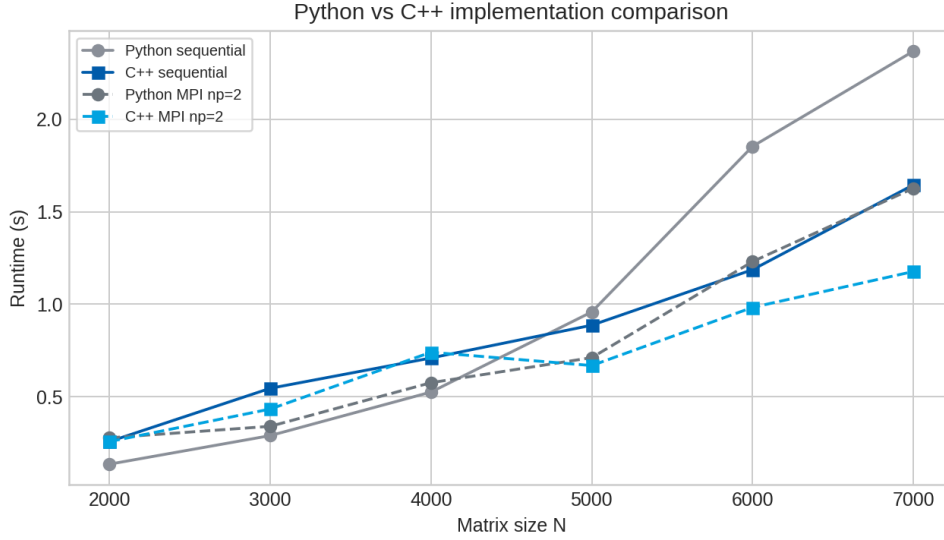


Figure 8: Python prototype versus C++ implementation on a two-node benchmark, 3-run averages.

The result is mixed at very small sizes because fixed overhead and implementation differences dominate. From $N = 5000$ onward in this two-node benchmark, the C++ MPI implementation becomes faster than the Python MPI prototype. Therefore, Python is kept only as a short validation and development reference; the analysis sections focus on the C++ solver.

12 Network and Wireless-Band Experiments

12.1 Two-Node Same-Band Wi-Fi versus Phone Hotspot

Before the three-machine router setup, the project also tested two-node Wi-Fi configurations. The same-band Wi-Fi benchmark used two VMs on the same Wi-Fi band with $np=2$. A phone-hotspot benchmark used two VMs through a different access link and was run for smaller sizes.

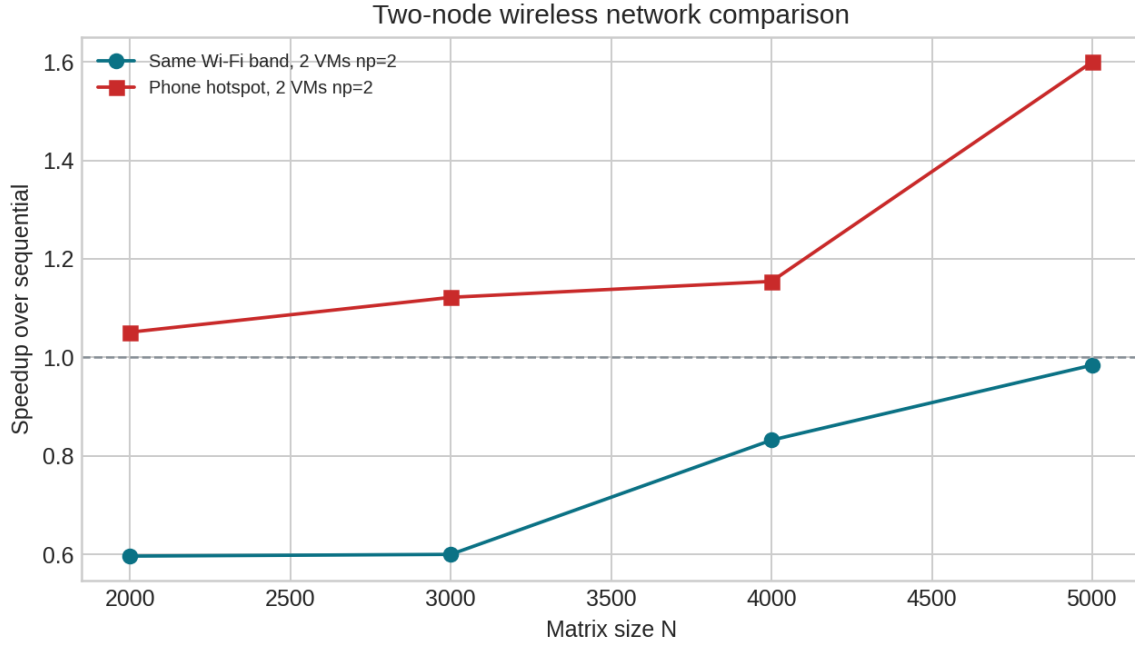


Figure 9: Two-node wireless topology comparison on overlapping sizes.

N	Same-band speedup	Same-band MPI (s)	Hotspot speedup	Hotspot MPI (s)
2000	0.596	0.824	1.051	0.681
3000	0.600	1.531	1.122	1.269
4000	0.832	2.010	1.154	2.192
5000	0.984	2.601	1.600	2.639

Table 10: Same-band Wi-Fi and phone-hotspot comparison for two-node runs.

The conclusion is subtle. Same-band Wi-Fi had low ping before the benchmark, but MPI did not outperform sequential until around $N = 6000$. The phone-hotspot run showed better speedup for $N = 2000$ – 5000 , but it used a smaller test range and a different access link. Therefore, it should be treated as evidence that wireless topology matters, not as a universal claim that hotspot is better. A strict same-router same-band versus different-band comparison should be added if a controlled dual-band router experiment is run later.

12.2 Communication-Reduction Variants

The same-band Wi-Fi setup was also used to test communication variants. These do not change the router setup, but they answer an important question: since Allreduce is the bottleneck, does reducing column synchronization payload improve performance?

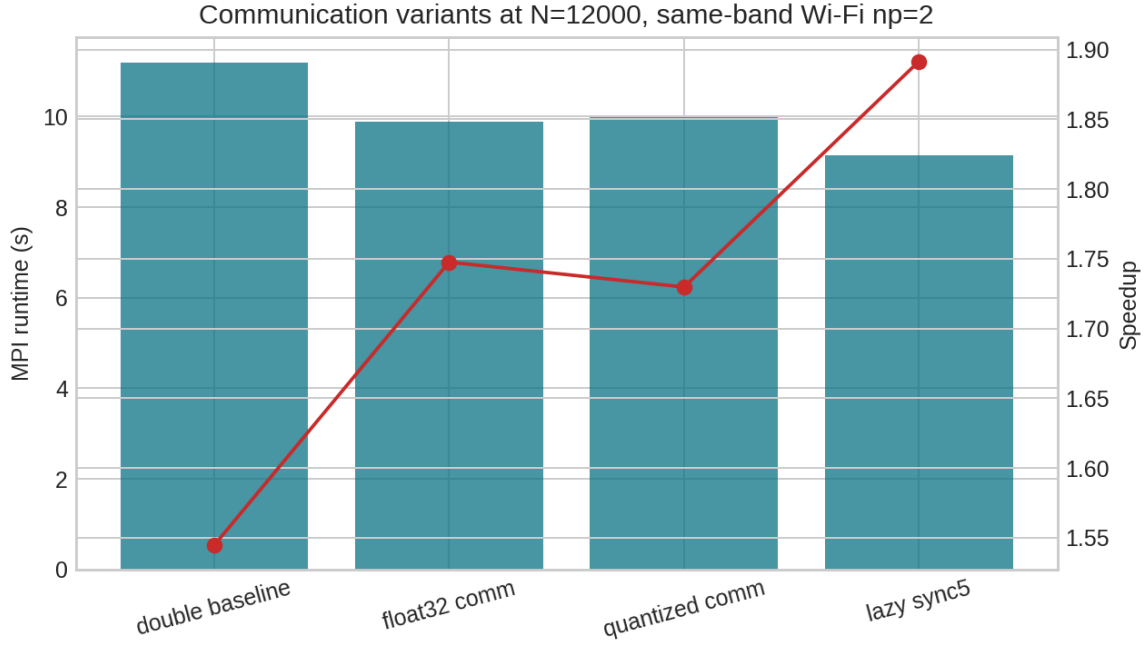


Figure 10: Communication variants at $N = 12000$, same-band Wi-Fi, $np=2$.

Variant	Payload ratio	MPI runtime (s)	Speedup
Double baseline	1.0	11.196	1.545
Float32 communication	0.5	9.895	1.748
Quantized communication	0.5	9.997	1.730
Lazy sync every 5 iterations	0.2	9.142	1.892

Table 11: Communication variant results at $N = 12000$.

Lazy synchronization gives the strongest speedup because it reduces the main column synchronization frequency to 20% of the baseline. Float32 communication is a safer compression-style optimization: it halves payload and improves runtime, with small objective differences around 10^{-10} – 10^{-11} . Quantization also halves payload but adds packing overhead and larger numerical error.

13 Four-Machine Wi-Fi Experiment

An earlier four-machine Wi-Fi run used $np=4$ on hosts 192.168.1.105, 192.168.1.227, 192.168.1.159, and 192.168.1.190. The run completed all 88 expected JSON files for sizes 2000–12000 and four repetitions.

N	Seq avg (s)	MPI np4 avg (s)	Speedup
4000	2.471	1.528	1.617
8000	8.881	3.451	2.574
10000	13.411	4.660	2.878
12000	19.673	6.143	3.203

Table 12: Four-machine Wi-Fi representative results.

This shows that adding a fourth machine can help for large N : at $N = 12000$, the speedup is about $3.20\times$. However, the later controlled router/Wi-Fi experiments focused on three machines because the four-machine Wi-Fi cluster was harder to keep stable after node/IP changes.

14 Dynamic Scheduling Experiments

14.1 Implemented Dynamic Modes

The dynamic branch extends the static row partition with chunk-based modes:

- **contiguous**: original static contiguous row ownership.
- **weighted-chunk equal**: split rows into chunks but use equal rank weights.
- **weighted-chunk adaptive**: use rank throughput from previous repetitions to suggest weights.
- **weighted manual 2,1,1**: manually assign more work to rank 0.
- **runtime queue**: experimental queue-based chunk assignment, tested only at small sizes due to overhead.

14.2 Does Dynamic Scheduling Improve Performance?

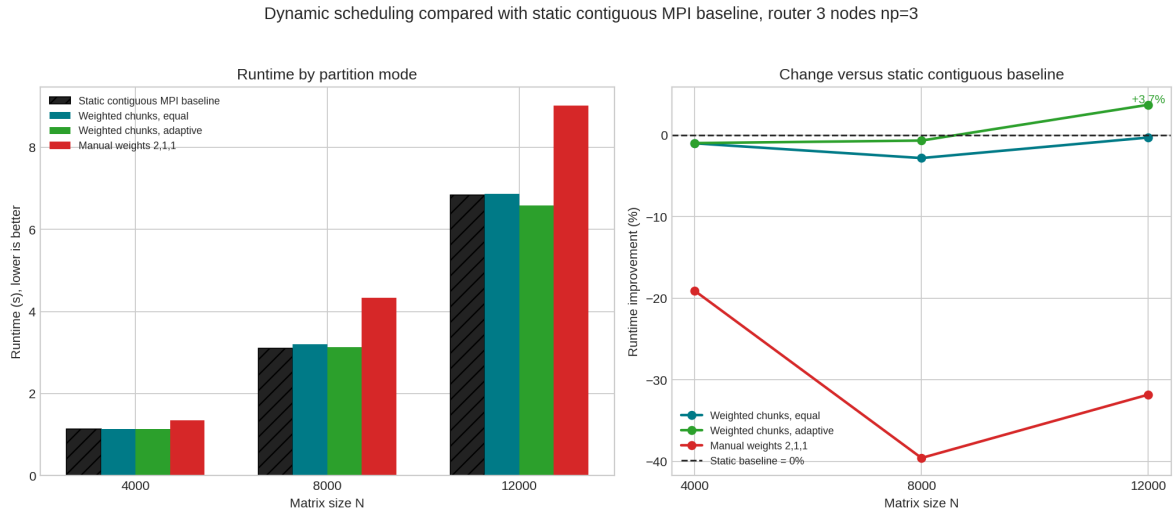


Figure 11: Dynamic partition modes on router 3 nodes, np=3. The static contiguous MPI partition is the baseline; it is already distributed across machines but assigns one fixed row interval to each rank.

N	Static contiguous MPI (s)	Weighted equal (s)	Adaptive (s)	Best result
4000	1.129	1.141	1.140	Contiguous
8000	3.108	3.195	3.128	Contiguous
10000	4.725	4.737	4.644	Adaptive by 1.7%
12000	6.839	6.858	6.584	Adaptive by 3.7%

Table 13: Dynamic scheduling summary on router 3 nodes, np=3.

Dynamic scheduling is not a universal improvement. At smaller sizes, chunk overhead and less cache-friendly ownership can cancel the benefit. At larger sizes, adaptive weights begin to help: at $N = 12000$, adaptive weighted chunks improve runtime by about 3.7% over contiguous.

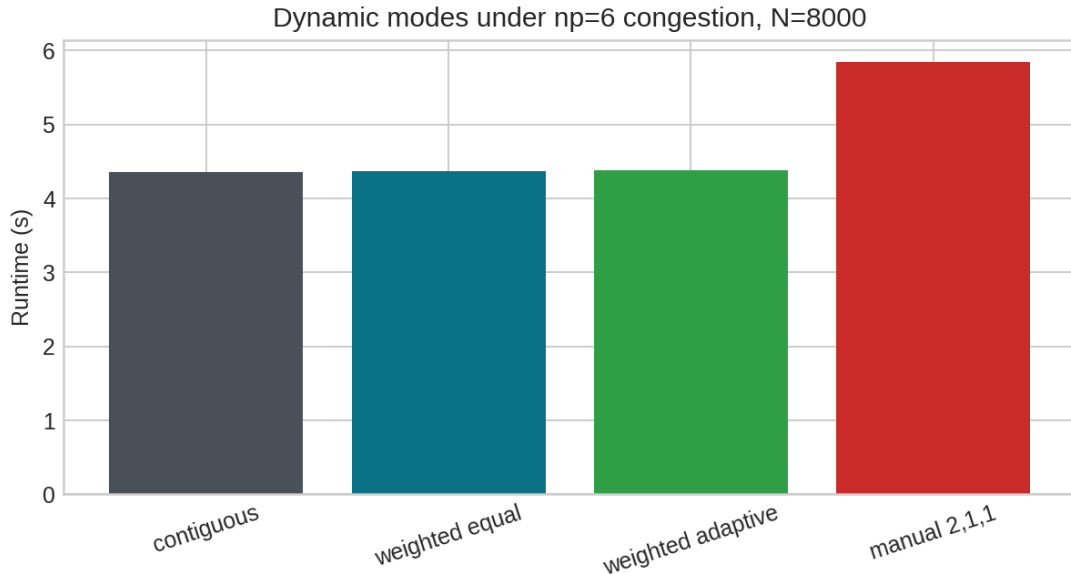


Figure 12: Dynamic modes under np=6 congestion at $N = 8000$.

Under the more congested np=6 setting at $N = 8000$, contiguous, weighted-equal, and weighted-adaptive are very close: 4.354 s, 4.367 s, and 4.382 s respectively. Manual 2,1,1 is much worse at 5.848 s because it over-assigns work to one rank. Runtime queue is correct as an experiment but far slower at small sizes: for example, around 2.18 s versus 0.58 s at $N = 2000$, and around 6.04 s versus 1.13 s at $N = 4000$. Therefore, the practical dynamic result is:

- Adaptive weighted chunks can improve large balanced runs slightly.
- Manual weights can hurt badly if the assumption about machine speed is wrong.
- Runtime queue is useful to demonstrate dynamic assignment but not suitable as the performance path in this implementation.

15 Discussion

15.1 What the Experiments Show

The experiments lead to four main conclusions.

First, the Sinkhorn solver has enough computation to benefit from MPI, but only after N is sufficiently large. Small runs are dominated by startup and communication overhead. Large runs at $N = 8000$ –12000 show clear speedup.

Second, the main scalability limiter is communication. The $2N = 16000$ process sweep peaks at np=4; np=8 is slower even though it has more processes. Communication-reduction variants support the same conclusion: reducing column synchronization payload/frequency improves runtime.

Third, router/Ethernet is the better demo setup because it is operationally more stable than Wi-Fi. Wi-Fi can still produce competitive runtimes, but it is more sensitive to route loss, band/access-point behavior, and VM host variability.

Fourth, dynamic scheduling is promising but not automatically better. The best adaptive result improves the large $N = 12000$ run by about 3.7%, but static contiguous remains excellent for many cases. Dynamic scheduling should be presented as an extension for heterogeneous clusters, not as a guaranteed replacement for static partitioning.

15.2 Limitations

- Dense matrix storage limits safe input size because memory grows as $O(N^2)$.
- Wi-Fi and VM scheduling introduce noise; repeated runs are necessary.
- The current dynamic runtime queue recomputes/dispatches too much and is not optimized.
- A fully controlled same-router same-band versus different-band experiment is not yet available; current wireless evidence compares same-band Wi-Fi with a phone-hotspot access link.

15.3 Future Work

Future work should include a true sparse kernel representation, a 2D block decomposition for larger clusters, non-blocking collectives, and a controlled dual-band router experiment. For dynamic scheduling, the next step is to keep weighted chunks but reduce overhead and learn weights over longer runs rather than relying on short repetitions.

16 Conclusion

The project implements and evaluates a real C++/MPI Sinkhorn optimal transport solver on a physical-machine VM cluster. It satisfies the main course requirements: at least three physical machines, C++ source code, MPI execution, sequential validation, decomposition explanation, communication analysis, load-balance analysis, and benchmark results.

The best stable three-node MPI results reach around $2.1 \times$ – $2.4 \times$ speedup for $N = 8000$ – 12000 . The four-node Wi-Fi run reaches about $3.20 \times$ speedup at $N = 12000$. The $2N = 16000$ speedup experiment shows that $\text{np}=4$ is better than $\text{np}=8$, confirming that Allreduce communication becomes the bottleneck.

The practical recommendation is to demo the project on the router/Ethernet three-machine setup, use $N = 8000$ with 2000 iterations for the two- to three-minute workload, and present dynamic weighted scheduling as an advanced extension that gives small gains on large heterogeneous runs but requires careful tuning.